Overview

Introduction

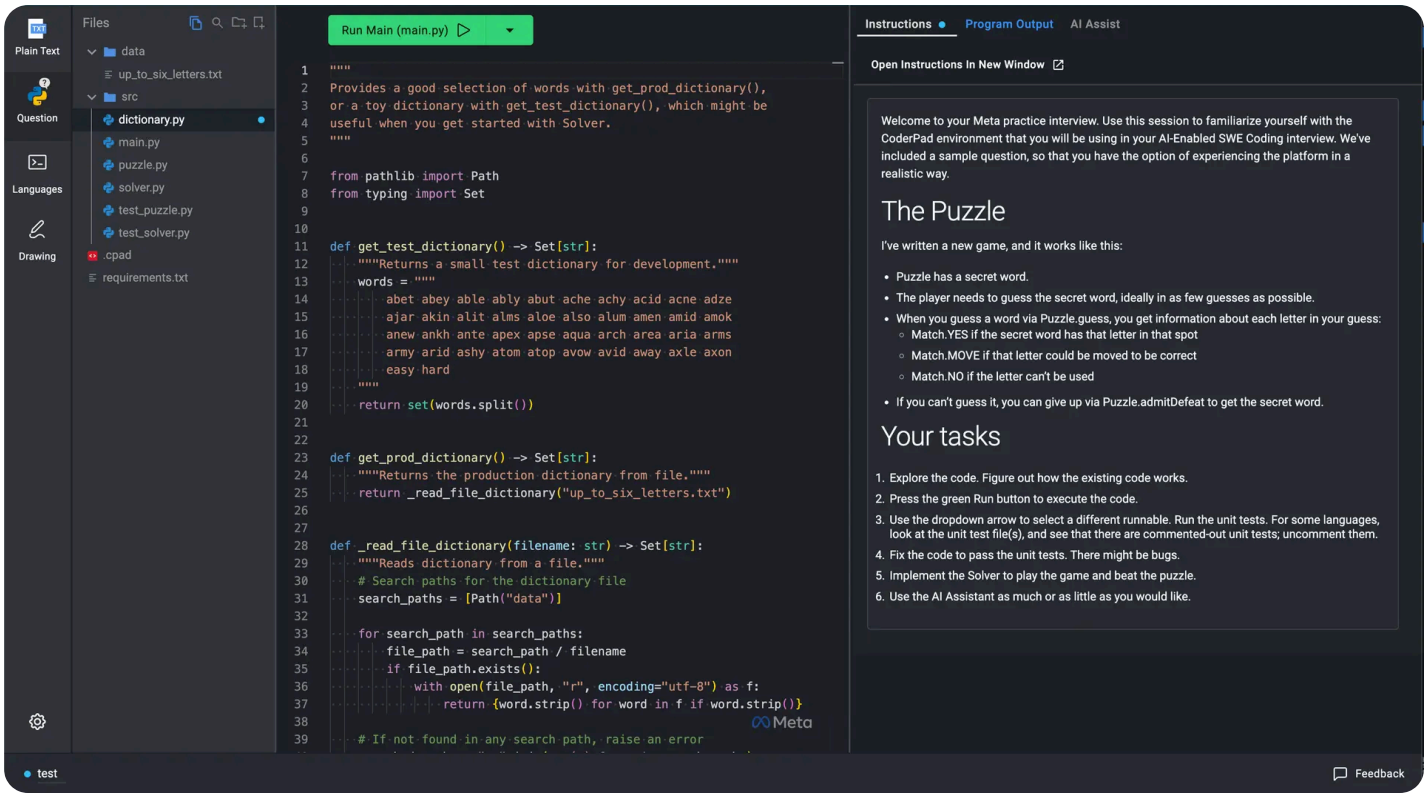
Understand what AI-enabled coding interviews are, why companies are adopting them, and what skills they actually evaluate.

We've spent the past few months interviewing dozens of engineers who recently went through AI-enabled coding interviews at Meta, Shopify, LinkedIn, Canva, and others. We wanted to know what actually happened, what they wished they'd known, and what separated the passes from the fails. The candidates who passed weren't better at prompting. They knew what to build, caught what the AI got wrong, and could defend every decision they made.

That's a different skill than what most people are training for, and this guide is built around it.

What makes this format different

The problems are much larger than traditional coding interviews. Instead of implementing a single function, you might be dropped into a multi-file codebase and asked to fix a bug, implement a feature, and optimize for scale, all in one session. Traditional interviews produce 30-50 lines of code. AI-enabled ones produce several hundred lines across multiple files. You're not writing all of it by hand, but you need to understand all of it.



A screenshot of the CoderPad environment from Meta

Algorithm memorization matters less. Reading unfamiliar code quickly matters a lot more. And there's a dynamic that doesn't exist in traditional interviews. You're managing two conversations simultaneously, one with the AI and one with the interviewer. Most candidates underestimate how unnatural this feels until they're in it. The **two main formats** have meaningfully different strategies and are covered in the next article.

What interviewers are evaluating

AI makes you *feel* productive even when you're failing. You paste the problem in, get 200 lines of code back, and feel like you're flying. But if you didn't plan, if you can't explain what was generated, if you're accepting everything uncritically, you're failing the interview while the code looks fine.

Interviewers aren't evaluating your prompting. They're evaluating whether you're in control. The rubric is consistent across companies:

1. **Problem-solving and approach** - Can you break down the problem and tackle things in the right order?
2. **Control over the AI** - Are you directing the AI, or is it directing you?
3. **Verification habits** - Do you review and test AI output, or accept it?
4. **Communication** - Can you keep the interviewer in the loop while working with AI?

Problem-solving and approach

The fundamentals haven't changed. Interviewers still want to know if you can understand a problem, break it down, and prioritize correctly.

AI does add a failure mode that doesn't exist in traditional interviews. Because the model responds instantly and produces complete-looking code, it can mask the fact that you don't have a plan. Strong candidates spend the first few minutes understanding the problem and forming an approach before they ever open the AI chat.

The most common failures here:

- Pasting the raw problem into AI without forming your own plan first, then building on whatever comes back
- Following the AI into architectural decisions that should be yours to make
- Chasing an AI-suggested rewrite when the model doesn't know the answer, which feels like momentum but is actually the model flailing

> Think out loud during this phase. Tell the interviewer what you're doing. Something like "I'm going to take a couple minutes to read through this and make sure I understand the requirements before I start." It signals confidence and keeps them engaged, the same way you would in a traditional coding interview.

Control over the AI

Interviewers across companies say the same thing. "We don't want the AI making decisions. We want to see you making decisions and using the AI to execute them." You decide the approach. You direct the AI to implement your plan. If the AI is making architectural decisions while you passively watch, you're already failing the evaluation.

A Rippling candidate received explicit feedback that they "relied too heavily on AI even though their initial approach was correct." They knew what to build but let the AI drive the implementation decisions instead of staying in control. At Canva, the interviewer pauses after each AI generation and asks "what does this code do?" If you can't walk through the generated logic confidently, it signals you're not actually directing the work.

The most common failures here:

- Accepting AI architectural suggestions without evaluating them (it proposes a BaseProcessor superclass; you add it without asking whether it actually makes sense)
- Treating AI output as the solution rather than a draft to review and accept or reject
- Not redirecting when the AI drifts into a different approach than the one you planned

> If you replaced the AI with a junior engineer pair programming with you, would you be comfortable with how you're directing them? That's what interviewers are looking for. You're the senior engineer. The AI is your very fast, occasionally wrong, pair partner.

Verification habits

AI will introduce bugs. It will make wrong assumptions about your data model, miss edge cases, and produce code that looks right but subtly isn't. Candidates who accept AI output without reviewing it leave a bad impression, even if the code happens to work. The review isn't just about catching mistakes. It's about showing the interviewer you're in control.

The most common failures here:

- Not running code after each generation, then discovering cascading failures at the end with no time left to fix them
- Skipping the line-by-line read because the generated code looked right at a glance
- Skipping tests because of time pressure, which almost always costs more time than it saves

Good verification means running the code after each meaningful change, reading through what the AI generated to confirm it matches what you asked for, and testing before moving to the next phase.

Communication

You're managing two conversations simultaneously. The challenge is that AI generates code faster than you can explain it, and there's a constant pull toward prompting without talking. Fight that instinct.

The most common failures here:

- Going silent for long stretches while prompting, leaving the interviewer with nothing to evaluate
- Narrating after the fact instead of before ("I just asked the AI to..." vs. "I'm going to ask the AI to...")
- Not explaining when you pivot ("the AI suggested DFS so I'm using DFS now") without saying whether you actually agree with it

State your intent before you prompt. Review output out loud. Flag anything that looks off.

How to use this guide

The Fundamentals section maps directly to the four evaluation areas above. **Codebase orientation** and **planning your approach** cover approach, **driving the AI** covers control, **verification and testing** covers exactly what it sounds like, and **communication** covers the narration piece. Read in order, or use the criteria above to identify your weak spot and jump straight there.

Read the rest of the Overview section next. It covers the **two main interview formats**, **how to prepare**, and **the patterns** companies actually test for (and why).

Once you've covered the fundamentals, check out the company-specific breakdowns. We currently have detailed posts for **Meta**, **Shopify**, and **LinkedIn**, with more being added as the format spreads. Each covers the platform, format specifics, and what that company's interviewers specifically look for.

> This is a fast-moving space. We update our content as we collect new data, but details shift between interview cycles. Always verify the current format with your recruiter.

Questions? Leave them in the comments. We read everything.